Universidad Mariano Gálvez de Guatemala · Desarrollo Web

PROYECTO FINAL

Sistema de Punto de Venta e Inventario

«Las Casas del Rincón Guatemalteco»

ESTUDIANTE
Andrés Ixcajoc López

CARNÉ
090-23-19115

CATEDRÁTICO
Jorge Mario Moya Marroquín

AGENDA

¿De qué vamos a hablar?

01 El negocio y el problema

02 Requerimientos del enunciado

03 Cómo se construyó

04 Modelo C4 y casos de uso

05 DDD: diseñar desde el negocio

06 Arquitectura hexagonal

07 Base de datos y MVC

08 El sistema funcionando

09 Diseño de interfaz (UI/UX)

10 Pruebas, Kanban e instalación

11 Demostración en vivo

01 · EL NEGOCIO

Un POS web que también lleva el inventario

El negocio vende productos de consumo a residentes de colonias y necesita **vender** y **controlar** lo que entra, lo que se vende y el dinero.

Pastas · Cárnicos · Embutidos · Lácteos · Golosinas · Aguas Gaseosas · Carbonatados

Cajero
Abre caja, despacha, cobra, factura, devuelve y hace el corte.

Administrador
Catálogos, compras, promociones, usuarios y reportes.

Cliente
Recibe la factura con QR y ve su promoción en el celular.

Core de Consumo Masivo
Sistema existente: provee productos, proveedores y precios.

02 · REQUERIMIENTOS

Todo lo que pide el enunciado tiene su pantalla

1 · Acceso
Login validado contra la base de datos

2 · Catálogos
Clientes, proveedores, productos y precios

3 · Despacho
Lectura por línea, subtotal y total

4 · Devoluciones
Por daño o vencimiento

5 · Cobro
Efectivo, tarjeta de crédito o débito

6 · Facturación
Datos del cliente y todos los productos

7 · Reportes
Cierre del día, devoluciones, clientes, compras

8 · QR
La factura trae la promoción para el celular

9 · Menú interactivo
Por módulos y según el rol

10 · Caja
Apertura con monto inicial y corte

10a · Promociones
Gratis, vale, 2.º a mitad, mayoreo, puntos

+ Inventario y Core
Compras, kardex y catálogo externo

03 · CÓMO SE CONSTRUYÓ

Primero diseñar, después programar

1. **Analizar** actores, procesos y reglas del enunciado
2. **Diseñar** modelo C4 y casos de uso
3. **Modelar el dominio** 10 contextos con DDD
4. **Puertos y adaptadores** un hexágono por módulo
5. **Programar** dominio → casos de uso → base de datos → pantallas
6. **Probar** 17 pruebas automáticas
7. **Kanban** Por hacer → En progreso → Terminado
8. **UI/UX** revisión, mockup en Figma y aplicación

Tecnologías libres y para la web

PHP 8.3 + Laravel 13
Lenguaje y framework web con MVC, rutas, validación y pruebas.

PostgreSQL 16
Base relacional con transacciones: una venta se guarda completa o no se guarda.

Docker
Levanta la base de datos igual en cualquier computadora.

Blade + Bootstrap 5
Pantallas web adaptables con la identidad «Jade & Maíz».

DomPDF y Simple QrCode
Factura y reportes en PDF; código QR de la promoción.

Git, GitHub, Projects y Figma
Versiones, Kanban y mockup de la interfaz.

Modelo C4: el sistema visto con zoom

1 · Contexto
El sistema, sus usuarios y el Core

2 · Contenedores
Web, vista móvil QR, Laravel y PostgreSQL

3 · Componentes
Los módulos dentro de Laravel

4 · Código
Las clases del despacho

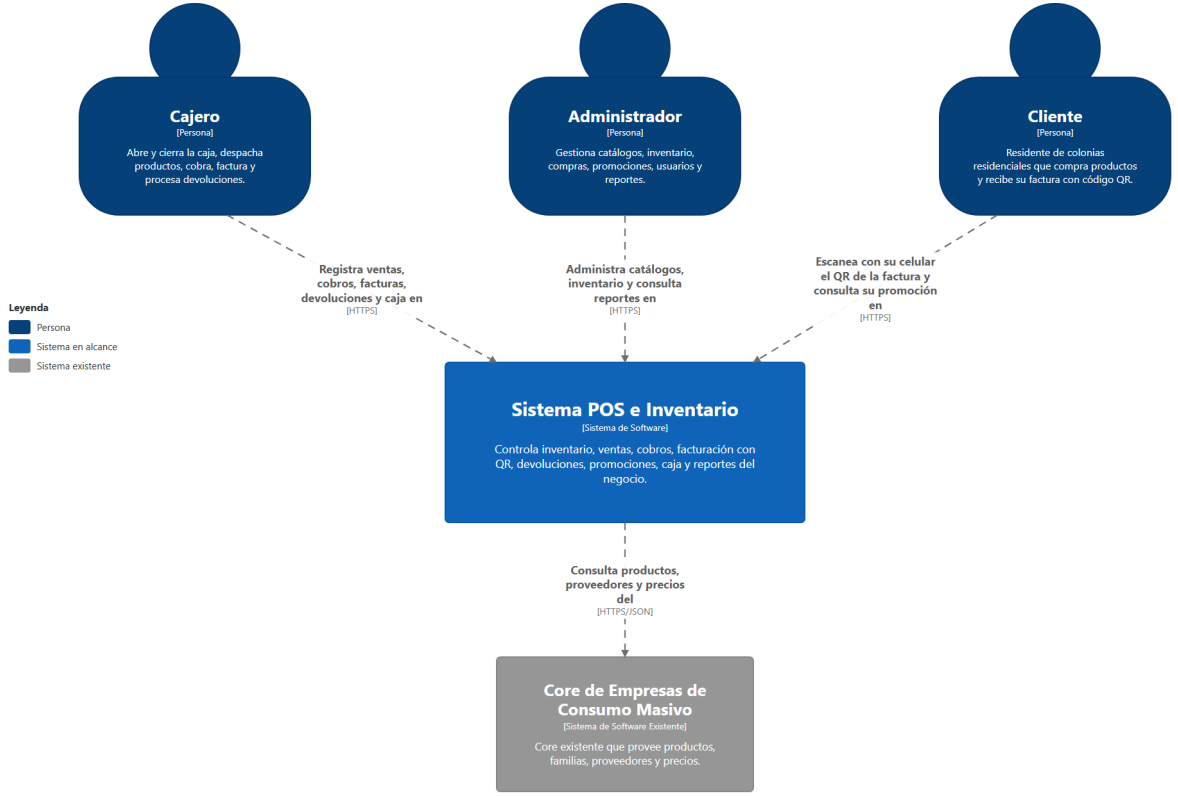


Diagrama de Contexto del Sistema – Las Casas del Rincón Guatemalteco

Nivel 1 del modelo C4: el sistema POS e Inventario, sus usuarios y el Core existente con el que se integra.

Contenedores y componentes

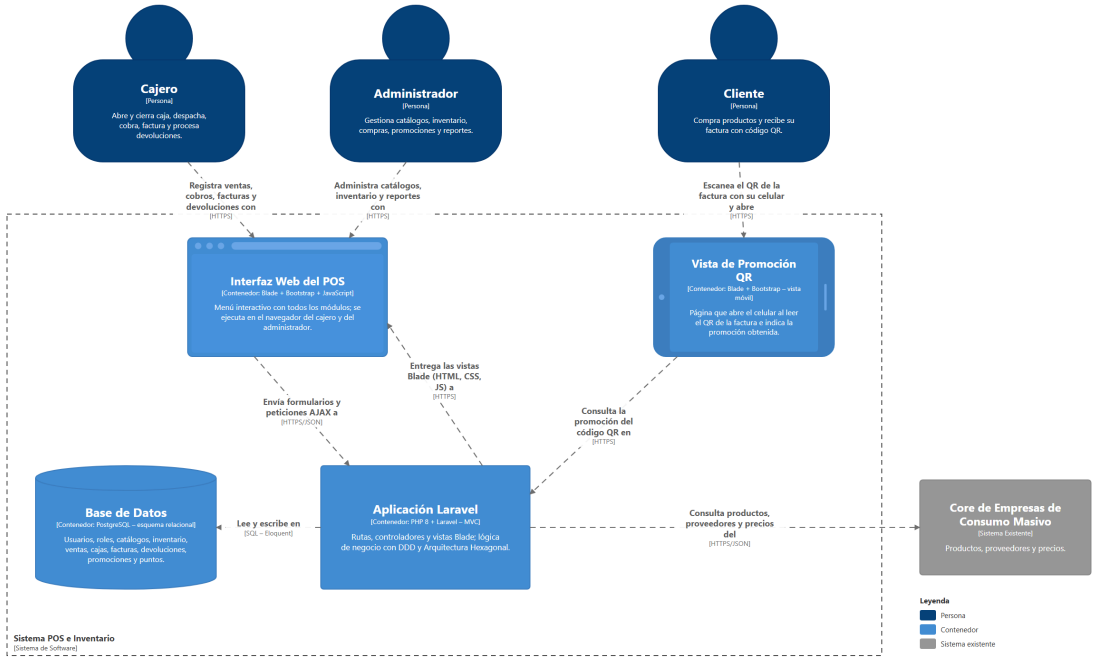


Diagrama de Contenedores – Las Casas del Rincón Guatemalteco

Nivel 2 del modelo C4: interfaces web, aplicación Laravel (MVC) y base de datos PostgreSQL que componen el Sistema POS e Inventario.

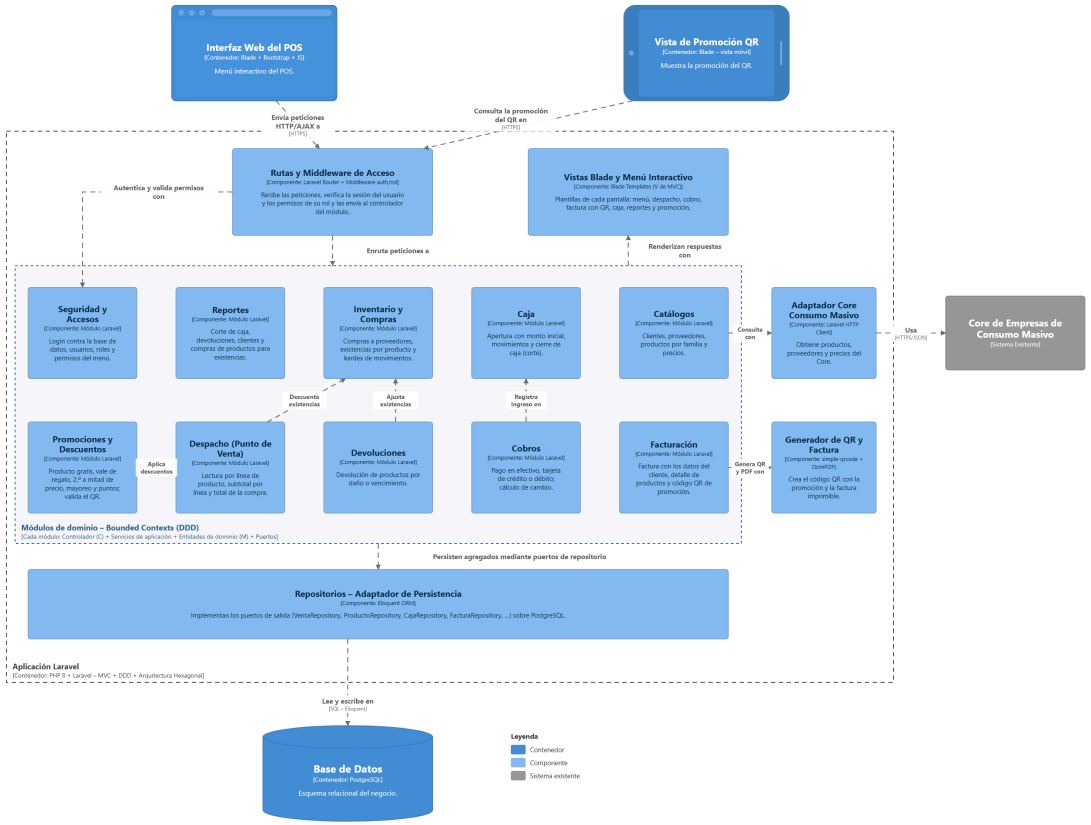


Diagrama de Componentes – Aplicación Laravel – Las Casas del Rincón Guatemalteco

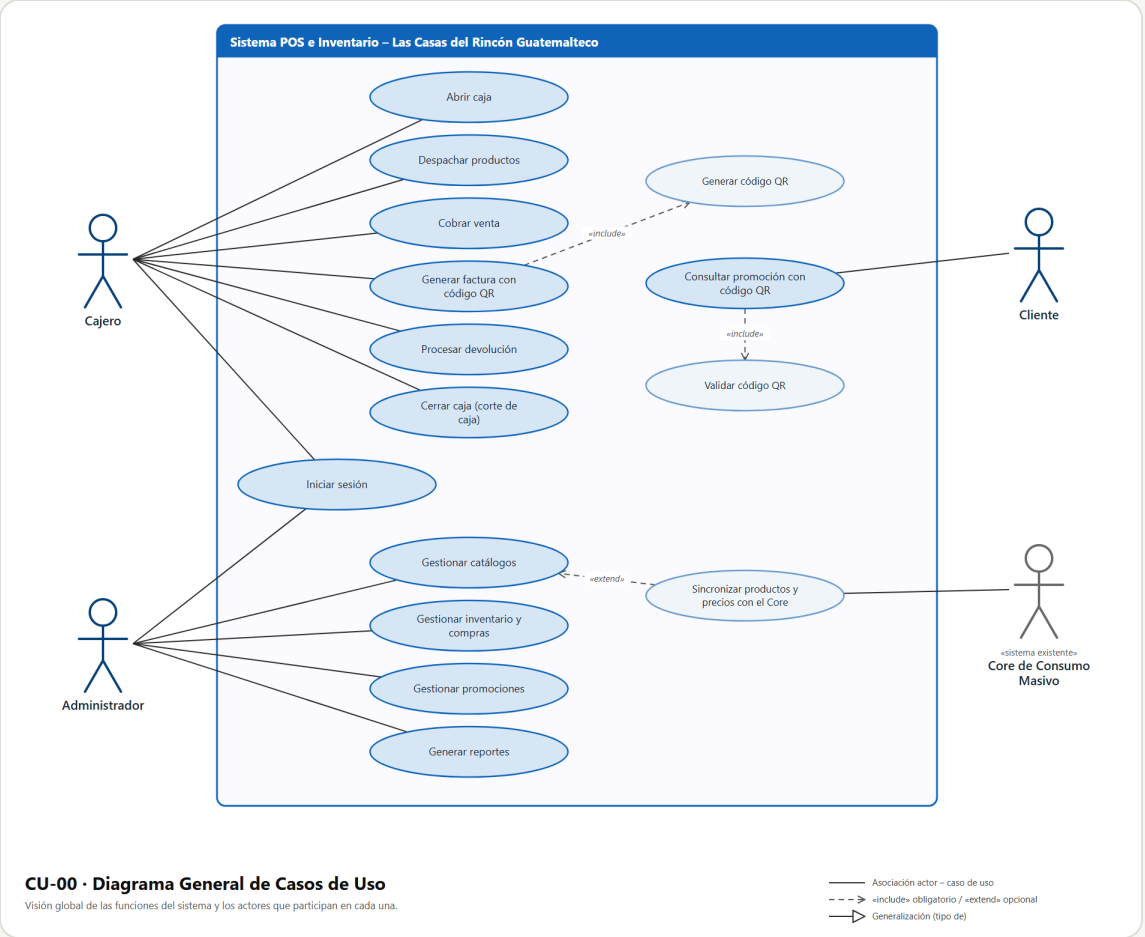
Nivel 3 del modelo C4: módulos (bounded contexts) y adaptadores de la aplicación Laravel, organizados con MVC, DDD y Arquitectura Hexagonal.

Casos de uso: qué hace cada actor

Un **caso de uso** es una acción del actor con el sistema.

- **«include»**: paso que siempre ocurre
- **«extend»**: paso opcional
- **Generalización**: variantes (3 pagos, 5 promociones)

6 diagramas en `docs/casos-de-uso`



DDD: el software tiene la forma del negocio

Antes de pensar en tablas o pantallas preguntamos: *¿qué hace el negocio y cuáles son sus reglas?*

Principal
Despacho, Promociones, Cobros, Facturación
donde el negocio gana dinero

Soporte
Catálogos, Inventario, Caja, Devoluciones
necesarios para operar

Genérico
Seguridad y Accesos, Reportes
comunes a cualquier sistema

05 · DDD

10 contextos delimitados

Seguridad
Usuario, rol, permiso

Catálogos
Cliente, proveedor, producto

Inventario
Compra, kardex

Caja
Apertura, movimientos, corte

Despacho
Venta y sus líneas

Promociones
Promoción, cupón, motor

Cobros
Pago y método

Facturación
Factura y QR

Devoluciones
Daño o vencimiento

Reportes
Solo lectura

Cada contexto tiene su vocabulario y sus reglas, como los departamentos de una tienda.

El código habla como el negocio

Negocio	Código
Abrir la caja	`AbrirCajaUseCase`
Agregar un producto	`Venta::agregarProducto()`
Canjear el cupón	`canjearCupon()`
Corte de caja	`Caja::cerrar()`
Motivo de devolución	`MotivoDevolucion::Dano`

Agregado: la Venta es «jefa» de sus líneas · **Servicio de dominio**: MotorPromociones

```
// app/Domain/Ventas/Venta.php — PHP puro
public function agregarProducto(..., int
$cantidad,
                                 int $existencia):
void
{
    $nueva = $this→cantidadDe($productoId) +
$cantidad;
    if ($nueva > $existencia) {
        throw new ReglaNegocioException(
            "Existencia insuficiente de
{$nombre}");
    }
    // suma a la línea existente o crea una nueva
}
```

Arquitectura hexagonal: puertos y adaptadores

Como un **enchufe**: el aparato define el enchufe (**puerto**) y cada país usa su **adaptador**.

Dominio

`app/Domain` · reglas del negocio, sin Laravel

Aplicación

`app/Application` · casos de uso y puertos

Infraestructura

`app/Infrastructure` · web, PostgreSQL, QR, PDF, Core

Infraestructura → Aplicación → Dominio



Recorrido real: el cajero escanea un producto

1. **DespachoController** adaptador primario: recibe el código de la pantalla
2. **AgregarProductoUseCase** puerto de entrada: la interfaz del caso de uso
3. **DespachoService** busca el producto por ProductoRepository (puerto de salida)
4. **Venta + MotorPromociones** dominio: valida existencias y calcula descuentos
5. **EloquentVentaRepository** adaptador secundario: guarda en PostgreSQL

El ensamblaje está en `HexagonalServiceProvider`: cada puerto se conecta con su adaptador.

La prueba: un puerto, dos adaptadores

El **Core de Consumo Masivo** se conecta con `CatalogoCorePort`.

CORE_DRIVER=archivo
Lee el catálogo exportado por el Core (catalogo.json)

CORE_DRIVER=http
Consume la API del Core

Capa anticorrupción
Traduce el formato del Core a nuestro modelo

Se cambia una línea del .env, no el dominio



07 · DATOS

Base de datos y MVC

PostgreSQL · migraciones de Laravel

usuarios · roles · permisos · clientes · proveedores · familias · productos · compras · movimientos_inventario · cajas · movimientos_caja · ventas · detalle_ventas · pagos · facturas · promociones · cupones · devoluciones

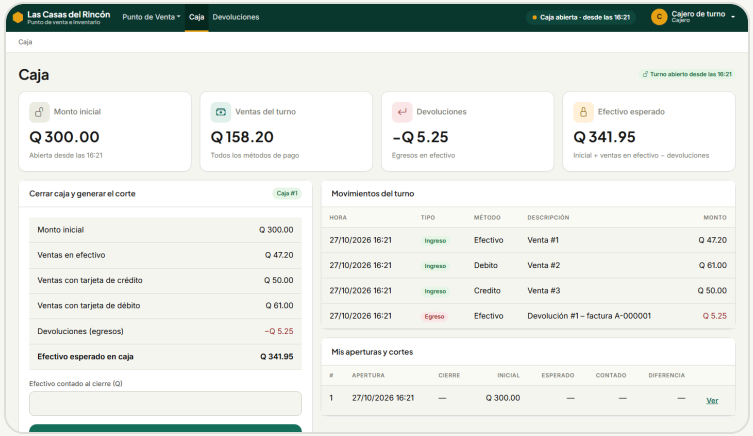
Al cobrar, inventario, pago y caja se guardan en una **transacción**: todo o nada.

Modelo
Reglas en Domain + tablas con Eloquent

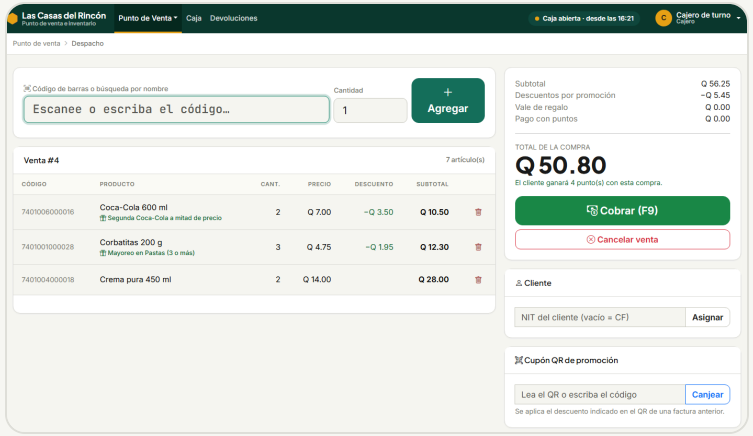
Vista
Plantillas Blade en resources/views

Controlador
Infrastructure/Http/Controllers

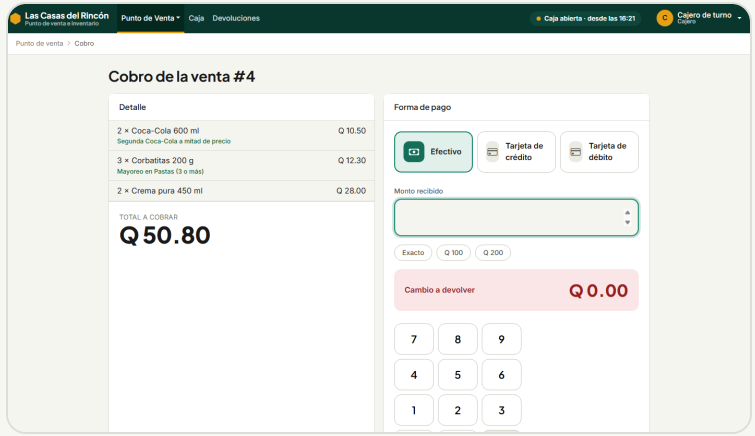
Una venta completa



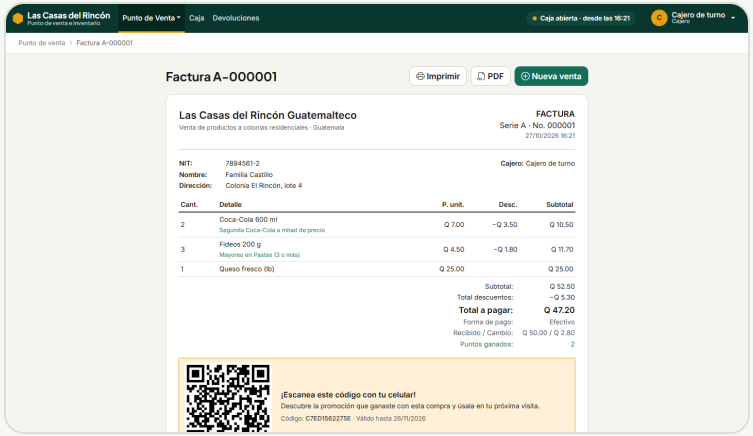
1 · Caja abierta



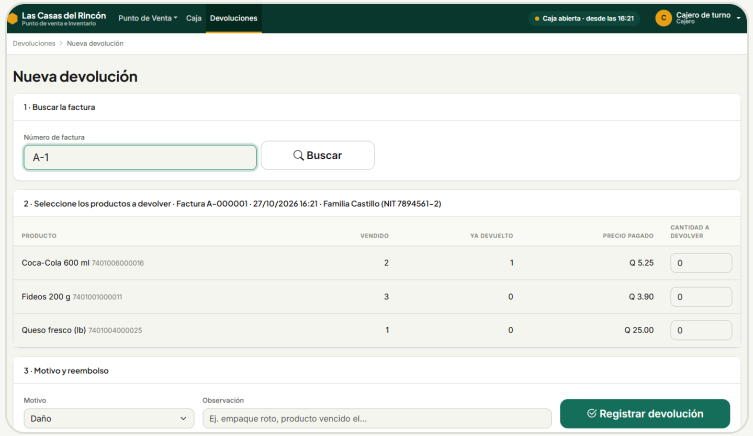
2 · Despacho por línea



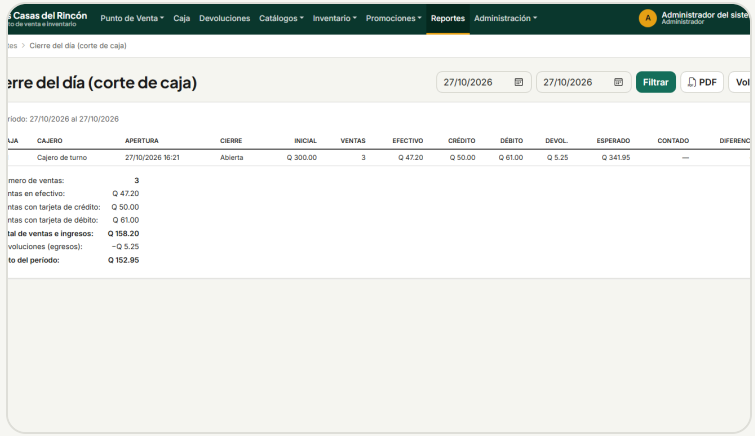
3 · Cobro y cambio



4 · Factura con QR



5 · Devolución



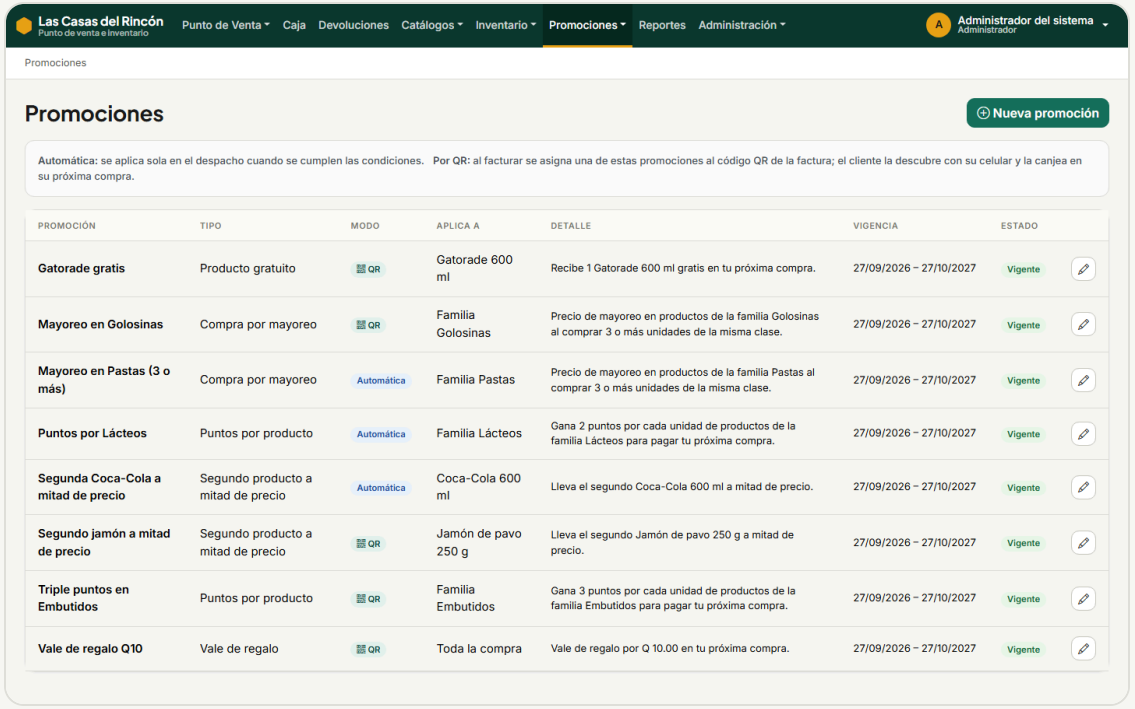
6 · Corte y reportes

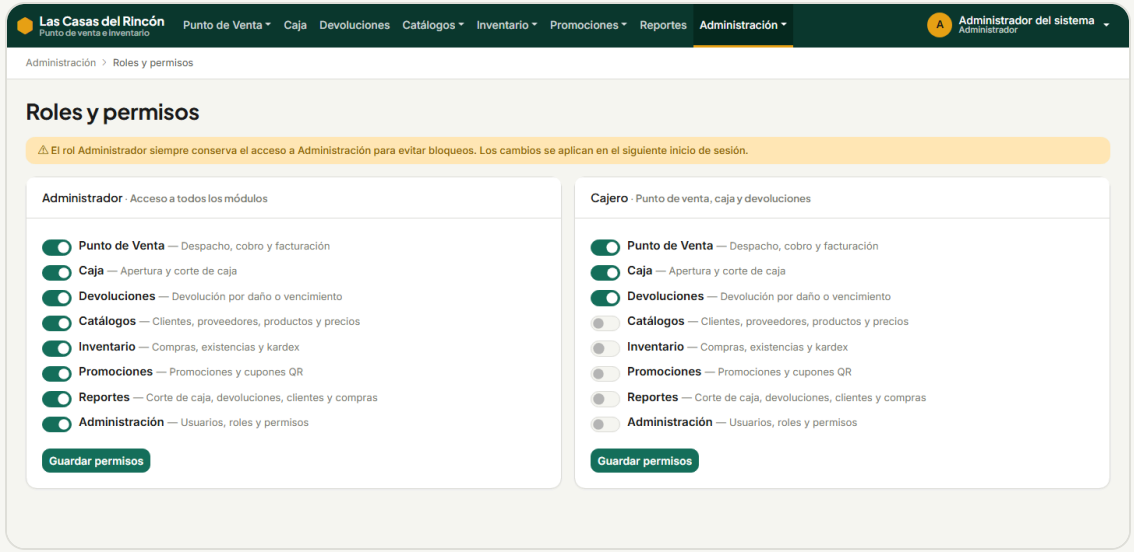
¿Cómo funciona el QR?

1. **Al facturar** se elige una promoción QR vigente y se crea un cupón único con vencimiento
2. **El QR** contiene la dirección /promocion/CÓDIGO
3. **El cliente** lo escanea y ve su promoción en el celular, sin iniciar sesión
4. **Próxima compra** el cajero canjea el código; el cupón queda usado

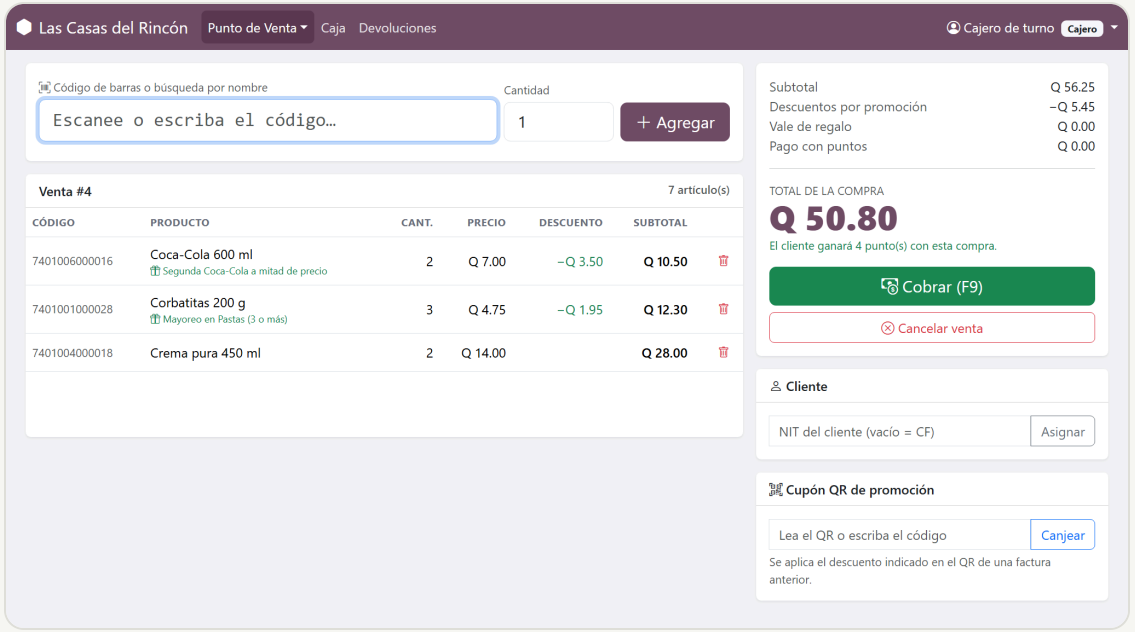


Promociones y administración

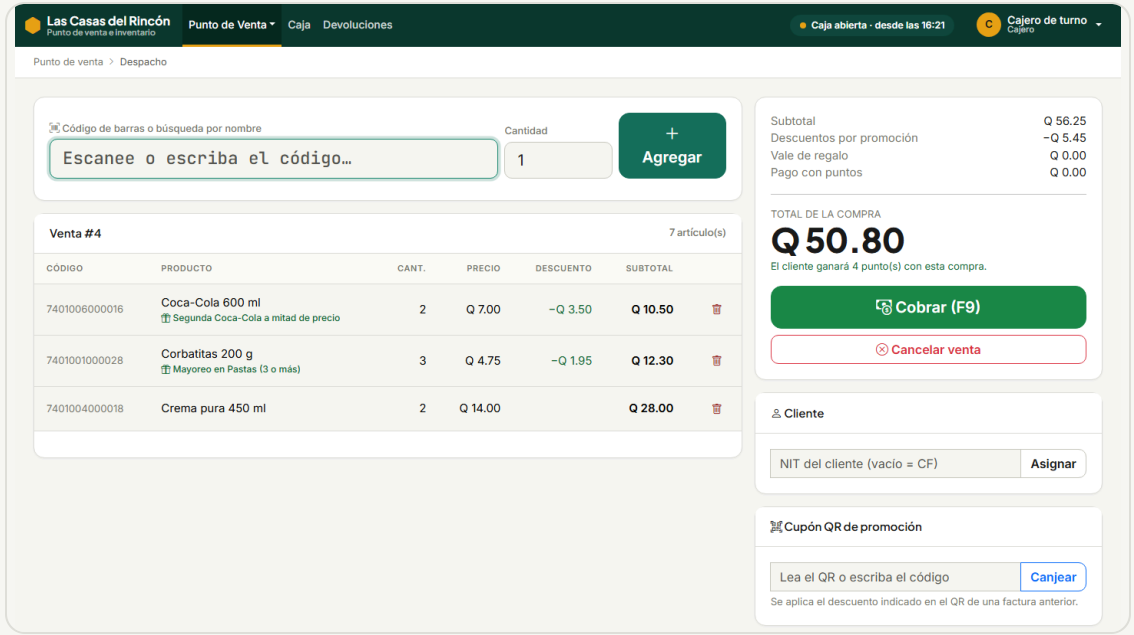




Diseño de interfaz: revisión → Figma → código



Antes



Después · «Jade & Maíz»

Estructura conocida

Módulos arriba, migas, listas y fichas

Identidad propia

Jade y maíz, tipografías e iconos propios

Mejor UX

Teclado de cobro, caja visible, estados con color

10 · CALIDAD

Pruebas, Kanban e instalación

17 pruebas · 147 verificaciones

Dominio, flujo completo de venta y todas las pantallas

Kanban en GitHub Projects

Cada tarea, una tarjeta: Por hacer → En progreso → Terminado

Montarlo en otra PC

```
git clone https://github.com/AndrewUMG/
          pos-las-casas-del-rincon.git
cd pos-las-casas-del-rincon
scripts\instalar.ps1 -Demo
scripts\iniciar.ps1        (-Red para el QR)
```

Requisitos: Git, Laragon y Docker Desktop · admin/admin123 · cajero/cajero123

11 · DEMO

Demostración en vivo

1. **Cajero** acceso, menú por rol y caja abierta
2. **Despacho** escanear 7401006000016 y ver la promoción
3. **Cobro y factura** efectivo con cambio, NIT 7894561-2, QR
4. **Celular** leer el QR y ver la promoción
5. **Devolución y corte** baja en kardex y diferencia de caja
6. **Administrador** catálogos, Core, promociones y reportes PDF

CONCLUSIONES

- Se cumplieron todos los requerimientos, verificados con pruebas automáticas.
- DDD puso las reglas del negocio en un solo lugar y con su mismo idioma.
- La arquitectura hexagonal separó el negocio de la tecnología.
- Diseñar primero y organizar con Kanban hizo el trabajo ordenado.

¡Gracias! ¿Preguntas?

https://github.com/AndrewUMG/pos-las-casas-del-rincon
https://github.com/users/AndrewUMG/projects/1